

Artificial Intelligence and Data Engineering for Orbital and Space Observation Systems

Laboratory 8: First Deep Learning Pipeline for Satellite Image Classification

Leszek Siwik
Faculty of Space Technologies
AGH University of Krakow

1 Introduction

In the previous laboratory, you built a first visual AI pipeline for satellite image classification. You used:

- image folders,
- feature extraction,
- classical machine learning models,
- prediction and visualization.

The model did not process images directly. Instead, every image was flattened into a long numerical vector.

In this laboratory, you will move one step further. You will build a first **deep learning** pipeline for image classification using PyTorch.

This means that instead of manually converting images into flattened feature vectors, you will use:

- image tensors,
- PyTorch Dataset classes,
- DataLoader objects,
- a convolutional neural network,
- training and evaluation loops.

The goal is to understand how satellite images can be prepared and processed by a neural network.

You will build the following pipeline:

```
download dataset -> prepare image folders -> create PyTorch Dataset  
-> create DataLoader -> train CNN -> evaluate -> predict -> visualize
```

2 Dataset Preview

In this laboratory, we continue using the EuroSAT RGB dataset based on Sentinel-2 satellite imagery. As before, you can preview the dataset here:

```
https://www.kaggle.com/datasets/apollo2506/eurosat-dataset
```

You do not need to download the dataset manually from Kaggle.
The dataset will be downloaded automatically using `torchvision`.
We will again use three selected classes:

```
Forest  
River  
Residential
```

These classes are useful because they represent visibly different types of Earth observation scenes:

- vegetation areas,
- water-related areas,
- human-made built-up areas.

3 Environment Setup

Install required libraries.

```
python3 -m pip install torch torchvision pillow numpy matplotlib scikit-learn
```

All commands in this laboratory should be executed from the root directory of your project.

4 Project Structure

The target project structure is as follows:

```
data/  
  raw/  
  processed/  
  inference_samples/  
  
models/  
reports/  
  
src/  
  data/  
    download_eurosat.py  
    prepare_image_dataset.py  
    create_noise_image.py  
  
  vision/  
    image_dataset.py  
    cnn_model.py  
    train_cnn.py  
    predict_cnn.py
```

Create the folders:

```
mkdir -p data/raw data/processed data/inference_samples
mkdir -p models reports
mkdir -p src/data src/vision
```

5 Task 1: Download Dataset

Create:

```
src/data/download_eurosat.py
```

Add the following code:

```
from pathlib import Path
from torchvision.datasets import EuroSAT

RAW_DATA_DIR = Path("data/raw")

def main():
    RAW_DATA_DIR.mkdir(parents=True, exist_ok=True)

    dataset = EuroSAT(
        root=str(RAW_DATA_DIR),
        download=True
    )

    print("=== EuroSAT Dataset Download ===")
    print(f"Downloaded dataset with {len(dataset)} images")
    print(f"Classes: {dataset.classes}")

if __name__ == "__main__":
    main()
```

Run:

```
python3 src/data/download_eurosat.py
```

or, if you use Python modules:

```
python3 -m src.data.download_eurosat
```

Expected Output

You should see something similar to:

```
=== EuroSAT Dataset Download ===
Downloaded dataset with 27000 images
Classes: ['AnnualCrop', 'Forest', 'HerbaceousVegetation', ...]
```

6 Task 2: Prepare Dataset

The original EuroSAT dataset contains several land-cover classes.
For this laboratory, we will use only three:

```
Forest
River
Residential
```

The goal is to create the following folder structure:

```
data/processed/images/
  train/
    forest/
    river/
    residential/
  test/
    forest/
    river/
    residential/
```

Each class should contain:

- 80 training images,
- 20 testing images.

This gives:

- 100 images per class,
- 300 images in total,
- 240 training images,
- 60 testing images.

The dataset is intentionally small to make CNN training fast.

Task 2.1: Create the Dataset Preparation Script

Create:

```
src/data/prepare_image_dataset.py
```

At the beginning of the script, import the required libraries:

```
from pathlib import Path
from PIL import Image
import random
import shutil
```

Task 2.2: Define Configuration

Add the following configuration:

```
RAW_ROOT = Path("data/raw")
PROCESSED_ROOT = Path("data/processed/images")

SELECTED_CLASSES = {
    "Forest": "forest",
    "River": "river",
```

```

    "Residential": "residential"
}

IMAGES_PER_CLASS = 100
TRAIN_RATIO = 0.8
TARGET_SIZE = (64, 64)
RANDOM_SEED = 42

SUPPORTED_EXTENSIONS = [".jpg", ".jpeg", ".png"]

```

This means that:

- RAW_ROOT points to the downloaded dataset,
- PROCESSED_ROOT points to the cleaned dataset,
- SELECTED_CLASSES maps original EuroSAT class names to lowercase folder names,
- IMAGES_PER_CLASS limits the dataset size,
- TARGET_SIZE ensures that all images have the same size,
- RANDOM_SEED makes the split reproducible.

Task 2.3: Find the EuroSAT Folder

Depending on the operating system and library version, the dataset may be stored in slightly different folders.

Therefore, do not hardcode the exact path.

Add:

```

def find_eurosat_folder(raw_root):
    for path in raw_root.rglob("*"):
        if path.is_dir():
            class_names = [
                child.name for child in path.iterdir()
                if child.is_dir()
            ]

            if all(
                class_name in class_names
                for class_name in SELECTED_CLASSES.keys()
            ):
                return path

    print("Error: Could not find EuroSAT class folders.")
    print("Make sure you ran src/data/download_eurosat.py first.")
    raise SystemExit(1)

```

Task 2.4: Prepare Output Folders

Before creating a new processed dataset, remove the old one if it exists.

Add:

```

def prepare_output_directories():
    if PROCESSED_ROOT.exists():
        shutil.rmtree(PROCESSED_ROOT)

```

```

for split_name in ["train", "test"]:
    for output_class in SELECTED_CLASSES.values():
        output_dir = PROCESSED_ROOT / split_name / output_class
        output_dir.mkdir(parents=True, exist_ok=True)

```

Task 2.5: Process One Image

Create a helper function that:

- opens one image,
- converts it to RGB,
- resizes it to 64x64,
- saves it in the processed dataset folder.

Add:

```

def process_single_image(input_path, output_path):
    with Image.open(input_path) as image:
        image = image.convert("RGB")
        image = image.resize(TARGET_SIZE)
        image.save(output_path)

```

Task 2.6: Prepare the Full Dataset

Add:

```

def prepare_dataset():
    random.seed(RANDOM_SEED)

    eurosat_folder = find_eurosat_folder(RAW_ROOT)
    prepare_output_directories()

    print("=== Preparing Image Dataset ===")
    print(f"EuroSAT folder: {eurosat_folder}")
    print(f"Processed dataset: {PROCESSED_ROOT}")

    for original_class, output_class in SELECTED_CLASSES.items():
        class_dir = eurosat_folder / original_class

        image_files = [
            path for path in class_dir.iterdir()
            if path.suffix.lower() in SUPPORTED_EXTENSIONS
        ]

        random.shuffle(image_files)
        selected_files = image_files[:IMAGES_PER_CLASS]

        split_index = int(TRAIN_RATIO * len(selected_files))

        train_files = selected_files[:split_index]
        test_files = selected_files[split_index:]

        print(f"\nClass: {original_class}")
        print(f"Selected images: {len(selected_files)}")

```

```

print(f"Training images: {len(train_files)}")
print(f"Testing images: {len(test_files)}")

for split_name, files in [("train", train_files), ("test", test_files)]:
    for index, input_path in enumerate(files):
        output_filename = f"{output_class}_{index:04d}.jpg"
        output_path = (
            PROCESSED_ROOT
            / split_name
            / output_class
            / output_filename
        )

        process_single_image(input_path, output_path)

```

Task 2.7: Add the Main Function

At the end of the script, add:

```

def main():
    prepare_dataset()

if __name__ == "__main__":
    main()

```

Task 2.8: Run the Script

Run:

```
python3 src/data/prepare_image_dataset.py
```

Expected Output

You should see output similar to:

```

=== Preparing Image Dataset ===
EuroSAT folder: data/raw/eurosat/2750
Processed dataset: data/processed/images

Class: Forest
Selected images: 100
Training images: 80
Testing images: 20

Class: River
Selected images: 100
Training images: 80
Testing images: 20

Class: Residential
Selected images: 100
Training images: 80
Testing images: 20

```

Task 2.9: Verify the Result

Check the generated folders:

```
ls data/processed/images/train/forest
ls data/processed/images/train/river
ls data/processed/images/train/residential

ls data/processed/images/test/forest
ls data/processed/images/test/river
ls data/processed/images/test/residential
```

Each training folder should contain 80 images.

Each testing folder should contain 20 images.

7 Task 3: Create PyTorch Dataset

In the previous laboratory, you manually converted each image into a flat vector.

In PyTorch, the usual approach is different.

We create a **Dataset** object.

A Dataset is responsible for:

- finding image files,
- assigning labels,
- loading images,
- applying transformations,
- returning one sample at a time.

Create:

```
src/vision/image_dataset.py
```

Add imports:

```
from pathlib import Path
from PIL import Image
from torch.utils.data import Dataset
from torchvision import transforms
```

Now create the dataset class:

```
class EuroSATDataset(Dataset):

    def __init__(self, root_dir, transform=None):
        self.root_dir = Path(root_dir)
        self.transform = transform

        if not self.root_dir.exists():
            print(f"Error: dataset directory does not exist: {self.root_dir}")
            print("Run src/data/download_eurosat.py first.")
            print("Then run src/data/prepare_image_dataset.py.")
            raise SystemExit(1)
```



```

self.samples = []

self.class_names = sorted([
    path.name
    for path in self.root_dir.iterdir()
    if path.is_dir()
])

self.class_to_index = {
    class_name: index
    for index, class_name
    in enumerate(self.class_names)
}

for class_name in self.class_names:

    class_dir = self.root_dir / class_name

    image_files = sorted([
        path for path in class_dir.iterdir()
        if path.suffix.lower()
        in [".jpg", ".jpeg", ".png"]
    ])

    for image_path in image_files:

        label = self.class_to_index[class_name]

        self.samples.append(
            (image_path, label)
        )

def __len__(self):
    return len(self.samples)

def __getitem__(self, index):

    image_path, label = self.samples[index]

    with Image.open(image_path) as image:

        image = image.convert("RGB")

        if self.transform is not None:
            image = self.transform(image)

    return image, label

```

What Happens Here?

This class reads images from folders such as:

```

data/processed/images/train/forest
data/processed/images/train/river
data/processed/images/train/residential

```

Folder names become class names.

Class names are mapped to integer labels.

For example, if the folders are sorted alphabetically:

```
forest
residential
river
```

then the mapping may be:

```
forest -> 0
residential -> 1
river -> 2
```

This mapping is created automatically by:

```
self.class_to_index
```

Task 3.1: Add Dataset Inspection Code

At the bottom of:

```
src/vision/image_dataset.py
```

add:

```
def main():
    transform = transforms.ToTensor()

    dataset = EuroSATDataset(
        root_dir="data/processed/images/train",
        transform=transform
    )

    print("=== PyTorch Dataset Inspection ===")
    print(f"Classes: {dataset.class_names}")
    print(f"Class mapping: {dataset.class_to_index}")
    print(f"Number of samples: {len(dataset)}")

    image_tensor, label = dataset[0]

    print(f"First image tensor shape: {image_tensor.shape}")
    print(f"First label: {label}")

if __name__ == "__main__":
    main()
```

Task 3.2: Run Dataset Inspection

Run:

```
python3 src/vision/image_dataset.py
```

or:

```
python src/vision/image_dataset.py
```

Expected Output

You should see something similar to:

```
=== PyTorch Dataset Inspection ===
Classes: ['forest', 'residential', 'river']
Class mapping: {'forest': 0, 'residential': 1, 'river': 2}
Number of samples: 240
First image tensor shape: torch.Size([3, 64, 64])
First label: 0
```

Important Explanation

The image tensor shape:

```
torch.Size([3, 64, 64])
```

means:

- 3 color channels: Red, Green, Blue,
- 64 pixels height,
- 64 pixels width.

This is different from the previous laboratory.

Previously, one image became a flat vector:

```
12288 numbers
```

Now, the image remains a 3D tensor:

```
channels x height x width
```

This format is used by convolutional neural networks.

8 Task 4: Create DataLoaders

A Dataset returns one image at a time.

During training, neural networks usually process images in small groups called **batches**.

A DataLoader is responsible for:

- batching samples,
- shuffling training data,
- loading data efficiently,
- preparing data for the training loop.

Create:

```
src/vision/train_cnn.py
```

Add imports:

```
from pathlib import Path

import torch
from torch.utils.data import DataLoader
from torchvision import transforms

from src.vision.image_dataset import EuroSATDataset
```

Define configuration:

```
TRAIN_DIR = Path("data/processed/images/train")
TEST_DIR = Path("data/processed/images/test")

BATCH_SIZE = 16
```

Add:

```
def create_dataloaders():
    transform = transforms.Compose([
        transforms.ToTensor()
    ])

    train_dataset = EuroSATDataset(
        root_dir=TRAIN_DIR,
        transform=transform
    )

    test_dataset = EuroSATDataset(
        root_dir=TEST_DIR,
        transform=transform
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=BATCH_SIZE,
        shuffle=True
    )

    test_loader = DataLoader(
        test_dataset,
        batch_size=BATCH_SIZE,
        shuffle=False
    )

    print("=== DataLoader Inspection ===")
    print(f"Training samples: {len(train_dataset)}")
    print(f"Testing samples: {len(test_dataset)}")
    print(f"Classes: {train_dataset.class_names}")
    print(f"Class mapping: {train_dataset.class_to_index}")

    images, labels = next(iter(train_loader))

    print(f"Batch image shape: {images.shape}")
    print(f"Batch label shape: {labels.shape}")
```

```
return train_loader, test_loader, train_dataset.class_names
```

Add temporary main function:

```
def main():
    create_dataloaders()

if __name__ == "__main__":
    main()
```

Run:

```
python3 -m src.vision.train_cnn
```

Expected Output

You should see something similar to:

```
=== DataLoader Inspection ===
Training samples: 240
Testing samples: 60
Classes: ['forest', 'residential', 'river']
Class mapping: {'forest': 0, 'residential': 1, 'river': 2}
Batch image shape: torch.Size([16, 3, 64, 64])
Batch label shape: torch.Size([16])
```

What Does This Mean?

The batch shape:

```
torch.Size([16, 3, 64, 64])
```

means:

- 16 images in one batch,
- 3 color channels,
- 64x64 pixels.

This is the standard input format for PyTorch image models.

9 Task 5: Create a Simple CNN Model

So far, in previous laboratories, you used classical machine learning models.

Those models required converting an image into:

```
one long vector of numbers
```

For example:

```
64 * 64 * 3 = 12288 features
```

This means:

- the image becomes a flat numerical vector,
- neighboring pixels lose spatial relationships,
- shape information becomes difficult to preserve.

This approach can work, but it has an important limitation:

Classical ML models usually do not explicitly understand that nearby pixels belong to the same visual structure.

CNNs solve this problem.

What Is a CNN?

CNN means:

Convolutional Neural Network

A CNN processes an image using small learnable filters.

Instead of analyzing the entire image at once, CNN looks at small local regions.

How Does a CNN Work?

A CNN usually processes data in stages:

```
image
convolution
activation
pooling
deeper convolution
classification
```

Step 1: Convolution

A small filter slides over the image.

At each position, the filter computes a weighted sum of nearby pixels.

This means that the CNN looks at a small local region of the image, for example:

```
Image patch (3x3):
[ 10  20  30 ]
[ 40  50  60 ]
[ 70  80  90 ]
```

Now suppose the CNN uses the following filter:

```
Filter (3x3):
[ 1  0 -1 ]
[ 1  0 -1 ]
[ 1  0 -1 ]
```

The filter is multiplied element-by-element:

```
(10 * 1) + (20 * 0) + (30 * -1)
+
(40 * 1) + (50 * 0) + (60 * -1)
+
(70 * 1) + (80 * 0) + (90 * -1)
```

which gives:

```
10 + 0 - 30
+
40 + 0 - 60
+
70 + 0 - 90
=
-60
```

The result:

```
-60
```

becomes one value in the feature map.

What Does It Mean?

This particular filter compares:

- left pixels,
- right pixels.

So it reacts strongly to:

- vertical edges,
- boundaries,
- strong intensity changes.

Example 1: No Edge

Suppose all neighboring pixels have similar intensity:

```
[ 50  50  50 ]
[ 50  50  50 ]
[ 50  50  50 ]
```

There is no strong visual change between left and right.

The filter output will be close to:

```
0
```

This means:

No important vertical boundary detected.

Example 2: Strong Vertical Edge

Now suppose the left side is dark and the right side is bright:

```
[ 10  10  200 ]
[ 10  10  200 ]
[ 10  10  200 ]
```

Now the filter detects a strong difference between:

- left pixels,
- right pixels.

The output becomes a large value.

This means that a strong vertical edge is present.

Visually, this may correspond to:

Dark area		Bright area
forest		river
#####		
#####		
#####		

When the filter slides across the entire image, CNN gradually builds a map of:

- where edges appear,
- where textures change,
- where objects begin and end.

This map is called:

```
feature map
```

For satellite imagery, such filters may help detect:

- coastlines,
- river boundaries,
- roads,
- building edges,
- field borders.

Step 2: Activation (ReLU)

After convolution, the filter produces numerical responses.

For example:

```
[-3.2, -1.1, 0.5, 2.8, 4.1]
```


Some values may be:

- negative,
- close to zero,
- strongly positive.

After that, CNN usually applies an activation function called:

ReLU

defined as:

$$f(x) = \max(0, x)$$

This means:

- negative values become 0,
- positive values stay unchanged.

Example:

Before ReLU:
[-3.2, -1.1, 0.5, 2.8, 4.1]

After ReLU:
[0.0, 0.0, 0.5, 2.8, 4.1]

This step acts like a feature selector.

Weak or irrelevant responses are suppressed:

negative response -> ignored

Strong responses are preserved:

positive response -> kept

For satellite imagery, this may mean:

- weak background textures are ignored,
- strong edges remain visible,
- important structures such as rivers, roads or building boundaries are preserved.

Without ReLU, the network would combine everything linearly, including weak or noisy responses.

With ReLU, the network can focus on the most informative visual patterns.

Step 3: Pooling

After convolution and activation, CNN often reduces the spatial size of feature maps. This operation is called:

Pooling

The most common type is:

Max Pooling

It looks at a small region, for example:

2 x 2 pixels

and keeps only the strongest response.

Example:

Input:

1	4
2	3

Max pooling selects:

4

because:

$\max(1, 4, 2, 3) = 4$

Now imagine a larger feature map:

1	4	2	0
2	3	1	5
7	1	6	2
0	8	3	4

After applying:

2x2 max pooling

the result becomes:

4	5
8	6

So:

4x4
becomes
2x2 feature map with the strongest signals

In your CNN, feature maps may shrink like:

```
64x64
```

```
32x32
```

```
16x16
```

For satellite imagery, this may mean:

- keeping strong coastlines,
- preserving road boundaries,
- keeping river structures,
- suppressing small pixel-level noise.

This helps:

- reduce computation,
- reduce overfitting,
- focus on important visual structures.

Unlike classical ML, CNN does not rely on manually designed image features. CNN learns features automatically.

This often leads to:

- better generalization,
- better spatial understanding,
- better robustness,
- better performance on visual tasks.

This is why CNNs are widely used in:

- satellite imagery,
- orbital inspection,
- planetary surface analysis,
- autonomous robotics,
- mission perception systems.

Create:

```
src/vision/cnn_model.py
```

Add:

```

import torch
from torch import nn

class SimpleCNN(nn.Module):

    def __init__(self, num_classes):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(
                in_channels=3,
                out_channels=16,
                kernel_size=3,
                padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(
                in_channels=16,
                out_channels=32,
                kernel_size=3,
                padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 16 * 16, 64),
            nn.ReLU(),
            nn.Linear(64, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)

        return x

```

What Happens Here?

The model has two parts:

- **features** — extracts visual patterns from images,
- **classifier** — assigns one of the output classes.

The image size changes inside the network:

```

Input:      3 x 64 x 64
After pool: 16 x 32 x 32
After pool: 32 x 16 x 16
Flatten:    8192 values
Output:     3 class scores

```

The model does not output class names directly.
It outputs class scores.
The highest score corresponds to the predicted class.

10 Task 6: Train the CNN

Return to:

```
src/vision/train_cnn.py
```

Add imports:

```
from torch import nn
from src.vision.cnn_model import SimpleCNN
```

Add configuration:

```
EPOCHS = 8
LEARNING_RATE = 0.001
MODEL_PATH = Path("models/cnn_model.pt")
CLASS_NAMES_PATH = Path("models/cnn_classes.txt")
```

Add device selection:

```
def get_device():
    if torch.cuda.is_available():
        return torch.device("cuda")

    return torch.device("cpu")
```

Add training function:

```
def train_model(model, train_loader, device):
    loss_function = nn.CrossEntropyLoss()

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=LEARNING_RATE
    )

    model.train()

    for epoch in range(EPOCHS):
        total_loss = 0.0

        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

            outputs = model(images)

            loss = loss_function(outputs, labels)

            loss.backward()
```

```

        optimizer.step()

        total_loss += loss.item()

    average_loss = total_loss / len(train_loader)

    print(f"Epoch {epoch + 1}/{EPOCHS}, Loss: {average_loss:.4f}")

```

Update main function:

```

def main():
    train_loader, test_loader, class_names = create_dataloaders()

    device = get_device()

    print(f"Using device: {device}")

    model = SimpleCNN(
        num_classes=len(class_names)
    )

    model = model.to(device)

    train_model(
        model,
        train_loader,
        device
    )

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.train_cnn
```

Expected Output

You should see something similar to:

```

Using device: cpu
Epoch 1/8, Loss: 1.0842
Epoch 2/8, Loss: 0.9123
Epoch 3/8, Loss: 0.7218
...

```

The loss should generally decrease.

It does not have to decrease perfectly in every epoch.

11 Task 7: Evaluate the CNN

Add the evaluation function to:

```
src/vision/train_cnn.py
```

```

def evaluate_model(model, test_loader, device):
    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)

            _, predicted = torch.max(
                outputs,
                dim=1
            )

            total += labels.size(0)

            correct += (
                predicted == labels
            ).sum().item()

    accuracy = correct / total

    print("=== Evaluation ===")
    print(f"Test samples: {total}")
    print(f"Accuracy: {accuracy:.4f}")

    return accuracy

```

Update main function:

```

def main():
    train_loader, test_loader, class_names = create_dataloaders()

    device = get_device()

    print(f"Using device: {device}")

    model = SimpleCNN(
        num_classes=len(class_names)
    )

    model = model.to(device)

    train_model(
        model,
        train_loader,
        device
    )

    evaluate_model(
        model,
        test_loader,
        device
    )

```

```
)

if __name__ == "__main__":
    main()
```

Run:

```
python3 -m src.vision.train_cnn
```

Expected Output

You should see:

```
=== Evaluation ===
Test samples: 60
Accuracy: 0.8500
```

The exact accuracy may differ.

12 Task 8: Save the Trained CNN

A trained neural network should be saved so it can be reused later.

Add:

```
def save_model(model, class_names):
    MODEL_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    torch.save(
        model.state_dict(),
        MODEL_PATH
    )

    with open(CLASS_NAMES_PATH, "w") as f:
        for class_name in class_names:
            f.write(class_name + "\n")

    print("=== Saving Model ===")
    print(f"Saved model: {MODEL_PATH}")
    print(f"Saved classes: {CLASS_NAMES_PATH}")
```

Update main function:

```
def main():
    train_loader, test_loader, class_names = create_dataloaders()

    device = get_device()

    print(f"Using device: {device}")

    model = SimpleCNN(
        num_classes=len(class_names)
    )
```



```

    model = model.to(device)

    train_model(
        model,
        train_loader,
        device
    )

    evaluate_model(
        model,
        test_loader,
        device
    )

    save_model(
        model,
        class_names
    )

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.train_cnn
```

Check:

```
ls models
```

You should see:

```

cnn_model.pt
cnn_classes.txt

```

13 Task 9: Predict a Single Image

Create:

```
src/vision/predict_cnn.py
```

Add imports:

```

from pathlib import Path

import torch
from PIL import Image
from torchvision import transforms
import matplotlib.pyplot as plt

from src.vision.cnn_model import SimpleCNN

```

Add configuration:

```
MODEL_PATH = Path("models/cnn_model.pt")
CLASS_NAMES_PATH = Path("models/cnn_classes.txt")
```

Add class loading function:

```
def load_class_names():
    if not CLASS_NAMES_PATH.exists():
        print(f"Error: class file not found: {CLASS_NAMES_PATH}")
        print("Train the model first.")
        raise SystemExit(1)

    with open(CLASS_NAMES_PATH, "r") as f:
        class_names = [
            line.strip()
            for line in f.readlines()
            if line.strip()
        ]

    return class_names
```

Add model loading function:

```
def load_model(class_names):
    if not MODEL_PATH.exists():
        print(f"Error: model file not found: {MODEL_PATH}")
        print("Train the model first.")
        raise SystemExit(1)

    model = SimpleCNN(
        num_classes=len(class_names)
    )

    state_dict = torch.load(
        MODEL_PATH,
        map_location="cpu"
    )

    model.load_state_dict(state_dict)

    model.eval()

    return model
```

Add prediction function:

```
def predict_image(model, class_names, image_path):
    path = Path(image_path)

    if not path.exists():
        print(f"Error: image not found: {path}")
        return

    transform = transforms.Compose([
        transforms.Resize((64, 64)),
        transforms.ToTensor()
    ])
```

```

with Image.open(path) as image:
    image = image.convert("RGB")
    image_for_plot = image.copy()
    image_tensor = transform(image)

image_tensor = image_tensor.unsqueeze(0)

with torch.no_grad():
    outputs = model(image_tensor)

    probabilities = torch.softmax(
        outputs,
        dim=1
    )

    confidence, predicted_index = torch.max(
        probabilities,
        dim=1
    )

predicted_class = class_names[
    predicted_index.item()
]

print("=== CNN Prediction ===")
print(f"Image: {image_path}")
print(f"Predicted class: {predicted_class}")
print(f"Confidence: {confidence.item():.4f}")

plt.imshow(image_for_plot)
plt.title(
    f"Prediction: {predicted_class}\nConfidence: {confidence.item():.4f}"
)
plt.axis("off")
plt.show()

```

Add main function:

```

def main():
    class_names = load_class_names()

    model = load_model(class_names)

    image_path = "data/processed/images/test/forest/forest_0000.jpg"

    predict_image(
        model,
        class_names,
        image_path
    )

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.predict_cnn
```

Expected Output

You should see something similar to:

```
=== CNN Prediction ===  
Image: data/processed/images/test/forest/forest_0000.jpg  
Predicted class: forest  
Confidence: 0.8732
```

A window should appear displaying the image and prediction.

14 Task 10: Test Unknown and Invalid Inputs

A classifier always tries to assign one of the known classes.

In our case, the known classes are:

```
forest  
residential  
river
```

This means that if we give the model an image outside these classes, it still predicts one of them. This is important in operational AI systems.

Experiment 1: Unknown EuroSAT Class

Modify:

```
image_path = "data/raw/eurosat/2750/Highway/Highway_1.jpg"
```

Run:

```
python3 -m src.vision.predict_cnn
```

Observe:

- What class is predicted?
- Is the confidence high or low?
- Does the prediction make sense?

Experiment 2: Random Noise Image

Create:

```
src/data/create_noise_image.py
```

Add:

```
from pathlib import Path  
import numpy as np  
from PIL import Image  
  
output_path = Path("data/inference_samples/noise.jpg")  
output_path.parent.mkdir(parents=True, exist_ok=True)
```

```
array = np.random.randint(
    0,
    256,
    size=(64, 64, 3),
    dtype=np.uint8
)

image = Image.fromarray(array)

image.save(output_path)

print(f"Saved noise image: {output_path}")
```

Run:

```
python3 -m src.data.create_noise_image
```

Then modify:

```
image_path = "data/inference_samples/noise.jpg"
```

Run:

```
python3 -m src.vision.predict_cnn
```

Observe:

- What class is predicted?
- Is the confidence meaningful?
- Why does the model not say “unknown”?

Important Insight

The model only knows the classes it was trained on.
It cannot automatically detect that an image is:

- unknown,
- invalid,
- random,
- outside the training distribution.

This can be dangerous in real systems.

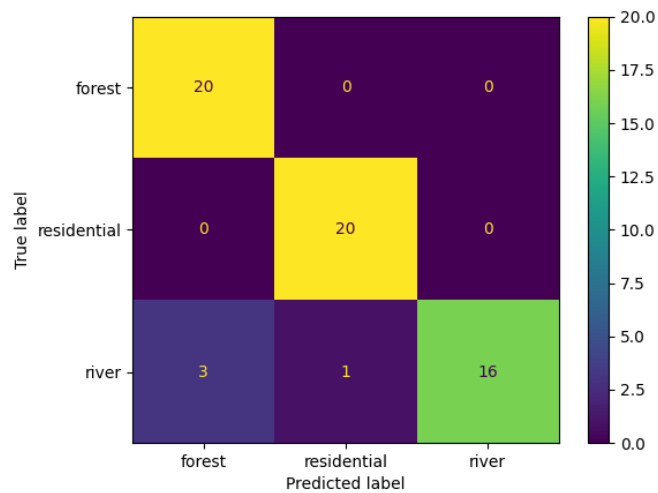
For example, an operational satellite AI system should not confidently classify meaningless or unfamiliar input as a valid land-cover class.

15 Task 11: Create a Confusion Matrix

Use predictions from the test set and create a confusion matrix.

You may use:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```



Save the plot to:

```
reports/confusion_matrix.png
```

What Should You See?

Your confusion matrix should look similar to:

How to Interpret It?

Each row represents:

```
true labels
```

Each column represents:

```
predicted labels
```

This means:

- diagonal values represent correct predictions,
- off-diagonal values represent classification mistakes.

Example interpretation:

- 20 in the forest diagonal means: all forest images were classified correctly.
- 20 in the residential diagonal means: all residential images were classified correctly.
- For river images:
 - 16 were classified correctly,
 - 3 were incorrectly classified as forest,
 - 1 was incorrectly classified as residential.

Engineering Interpretation

This matrix tells you not only:

overall accuracy

but also:

- which classes are easy to recognize,
- which classes are visually similar,
- where the model makes systematic mistakes,
- which classes may require more training data.

For satellite imagery this is especially important because:

- forests and rivers may share similar textures,
- residential areas may sometimes resemble industrial zones,
- some terrain classes may be visually ambiguous.

A confusion matrix helps engineers decide:

- whether the model is reliable enough,
- which classes need improvement,
- whether additional training data is required,
- whether another model architecture may help.

Answer:

- Which classes are confused most often?
- Is forest easier to classify than residential?
- Which errors would be most dangerous in a real system?

16 Task 12: CNN vs Classical Machine Learning

So far, you built two different image classification pipelines:

- a classical machine learning pipeline using flattened image vectors,
- a convolutional neural network (CNN) pipeline using learned visual features.

Both systems can classify satellite images.

However, an important engineering question remains:

What practical advantages does CNN provide compared to classical machine learning?

In real Earth Observation systems, CNN models require:

- more computation,
- longer training time,
- more implementation effort.

So before choosing CNN in a real mission system, an engineer should be able to justify:

- whether CNN actually improves performance,
- whether CNN improves robustness,
- whether the additional computational cost is justified.

Task

Use exactly the same dataset as before:

```
forest
residential
river
```

Use exactly the same:

- training images,
- testing images,
- train/test split.

Now compare:

- your classical ML model from previous laboratory,
- your CNN model from this laboratory.

For both models, measure:

- training time,
- final test accuracy,
- confusion matrix,
- one correctly classified image,
- one incorrectly classified image.

Save your analysis in:

```
reports/cnn_vs_ml.txt
```


Suggested Output Format

```
CNN VS CLASSICAL ML
=====
```

```
CLASSICAL ML:
Model: Random Forest
Training time: ...
Accuracy: ...
```

```
CNN:
Model: Simple CNN
Training time: ...
Accuracy: ...
```

```
BETTER ACCURACY:
...
```

```
FASTER TRAINING:
...
```

```
FEWER CLASS CONFUSIONS:
...
```

```
GENERALIZATION:
...
```

Engineering Analysis

Answer the following questions:

- Which model achieved higher accuracy?
- Which model trained faster?
- Which model produced fewer class confusions?
- Which model handled difficult images better?
- Which model appears more robust?
- Is the additional computational cost of CNN justified?

What Should You Observe?

Classical machine learning usually sees an image as:

```
one long vector of numbers
```

For example:

```
64 * 64 * 3 = 12288 features
```

This means that:

- neighboring pixels lose spatial meaning,
- edges and textures are not explicitly modeled,

- shape information is difficult to learn.

CNN works differently.

Instead of flattening the image immediately, CNN processes local regions using convolution filters.

This allows CNN to learn:

- edges,
- textures,
- shapes,
- spatial structures,
- repeating visual patterns.

This is especially important in satellite imagery because many classes differ by:

- texture patterns,
- spatial organization,
- object distribution,
- land-cover geometry.

Final Reflection

Based on your experiments, answer:

- What does CNN learn that classical ML may miss? Support your answer with examples from the dataset used in this laboratory.
- When might classical ML still be preferable?
- When would CNN be the better engineering choice?
- Would you deploy CNN in a real orbital observation system? Why?

17 Task 13: Add Data Augmentation

In real computer vision systems, the amount of labeled training data is often limited.

One way to improve model robustness is:

`data augmentation`

Data augmentation means:

Creating slightly modified versions of existing training images, without changing their semantic meaning.

For example, a forest is still a forest if:

- the image is slightly rotated,
- the image is flipped,

- the camera angle changes slightly,
- the object appears in a slightly different orientation.

Instead of collecting new satellite images, we artificially generate additional visual variation. Modify the training transform in:

```
src/vision/train_cnn.py
```

Use:

```
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor()
])
```

What Happens Here?

This transform introduces random visual changes:

- `RandomHorizontalFlip()`
 - randomly flips the image left-to-right,
 - helps the model learn orientation invariance.
- `RandomRotation(10)`
 - randomly rotates the image by up to 10 degrees,
 - helps the model handle slight viewpoint variations.
- `ToTensor()`
 - converts the image into a PyTorch tensor.

This means that during training, the model may see slightly different versions of the same image in different epochs.

For example:

- epoch 1: original forest image
- epoch 2: rotated forest image
- epoch 3: flipped forest image

The label stays the same:

```
forest -> forest
```

but the visual appearance changes.

Why Is This Useful?

Without augmentation, the model may memorize the training images.

With augmentation, the model learns more general visual patterns such as:

- textures,
- edges,
- shapes,
- spatial structures.

This often improves:

- generalization,
- robustness,
- resistance to overfitting.

This is especially useful for satellite imagery because:

- satellites observe terrain from different angles,
- orbital geometry changes over time,
- lighting conditions may vary,
- sensor orientation may differ between acquisitions.

Task

Retrain the model using augmented data.

Then compare:

- training loss,
- final accuracy,
- confusion matrix.

Finally, answer:

- Did augmentation improve accuracy?
- Did training become more stable?
- Which classes improved the most?
- Why can augmentation be useful for satellite imagery?

18 Task 14: Train for More Epochs

Change:

```
EPOCHS = 8
```

to:

```
EPOCHS = 20
```

Then retrain.

Answer:

- Did accuracy improve?
- Did loss continue to decrease?
- Is more training always better?

Task 15: Add a Third Convolutional Layer

Modify:

```
src/vision/cnn_model.py
```

Add another convolutional block:

```
nn.Conv2d(32, 64, kernel_size=3, padding=1),  
nn.ReLU(),  
nn.MaxPool2d(2)
```

Remember that after three pooling layers, the spatial size becomes:

```
64 -> 32 -> 16 -> 8
```

So the classifier input should become:

```
64 * 8 * 8
```

Answer:

- Did the deeper model improve performance?
- Did training become slower?
- What is the trade-off between model size and performance?

19 Submission

Submit the following artifacts:

- dataset download script,
- dataset preparation script,
- PyTorch Dataset script,

- CNN model script,
- training script,
- prediction script,
- trained CNN model,
- class name file,
- noise image script, if completed,
- confusion matrix plot,
- CNN vs classical ML comparison report,
- augmentation / epoch / architecture experiment results.
- short reflection answering the following questions:
 - What is the difference between a flattened image vector and an image tensor?
 - What does a PyTorch Dataset do?
 - What does a DataLoader do?
 - Why are CNNs useful for image data?
 - What does the model output before the final class is selected?
 - Why can the model classify random or unknown images as one of the known classes?
 - Why can this behavior be dangerous in real orbital AI systems?
 - What would need to be improved before using such a model operationally?
- repository link.

20 Final Note

This laboratory introduces the first real deep learning pipeline in this course.
The most important conceptual shift is:

Images are no longer treated only as flat vectors. They become structured tensors processed by convolutional neural networks.

This is the foundation of many modern AI systems used in:

- satellite image classification,
- land cover recognition,
- disaster monitoring,
- Earth observation analytics,
- onboard visual intelligence,
- orbital perception systems.